



FDE 実践研修プログラム

Forward Deployed Engineer を育てる、レベル別・実践型カリキュラム

対象:新卒 / 中途 / 未経験者 | 教材:社内情報検索アプリ(company_inner_search_app2)

このスライドの使い方

研修講師・受講者の両方が参照できる説明資料です

対象読者

- **研修講師・メンター**:カリキュラムの説明・進行に使用
- **受講者**:研修全体の見取り図として、初日のオリエンテーションで配布
- **マネージャー**:育成計画・アサイン判断の参考資料として

操作方法

- ページ送り:← → キー、スペースキー、画面右下のボタン
- **ブラウザの印刷機能で PDF 化**できます(1スライド=1ページ)
- ファイルを開くだけで動作します(ネット接続不要)

■ 用語メモ — このボックスについて

用語メモ 各スライドの下部にあるこの水色のボックスでは、そのスライドに登場する**専門用語**を**初心者向けに解説**しています。分からない言葉が出てきたら、まずここを確認してください。巻末には全用語をまとめた「用語集」もあります。

FDE Forward Deployed Engineer(フォワード・デプロイド・エンジニア)の略。「顧客の現場に前方展開(forward deploy)するエンジニア」という意味。詳しくは次のセクションで説明します。

アジェンダ

1. FDE という仕事を知る

FDE の定義・働き方・求められる 3 つのスキル

2. 研修の全体設計

ゴール・修了判定基準・レベル別の 3 コース・教材アプリの紹介

3. 未経験者向けコース(16週間)

プログラミングの基礎から総合演習まで、4 フェーズの詳細

4. 新卒向けコース(12週間)

実務レベルの開発プラクティスとコンサルスキルの習得

5. 中途向けコース(6週間)

アセスメント → 出身別ブリッジ研修 → 独り立ち判定

6. 研修の運営設計

評価制度・メンター制度・卒業後フォロー

7. 付録:用語集

IT 基礎・生成 AI・コンサル・開発プラクティスの用語を一覧で解説

SECTION 1

FDE という仕事を知る

まずは「FDE とは何者か」「普通のエンジニアやコンサルタントと何が違うのか」を理解します。
ここが全研修の出発点です。

FDE(Forward Deployed Engineer)とは

「顧客の現場に入り込んで、技術で課題を解決するエンジニア」

顧客のオフィスや現場に自ら出向き(= forward deploy)、顧客と一緒に課題を見つけ、その場でソフトウェアを作
って解決する職種。

従来のエンジニア

自社オフィスで、決められた仕様書どおりに
開発する。顧客と直接話す機会は少ない。

従来のコンサルタント

顧客の課題を分析し提案資料を作るが、実際
にモノを作るのは別のエンジニア。

FDE

提案から実装までを一人(一チーム)で担う。
「話せるエンジニア」「作れるコンサル」の
ハイブリッド。

■ 用語メモ

Forward Deploy

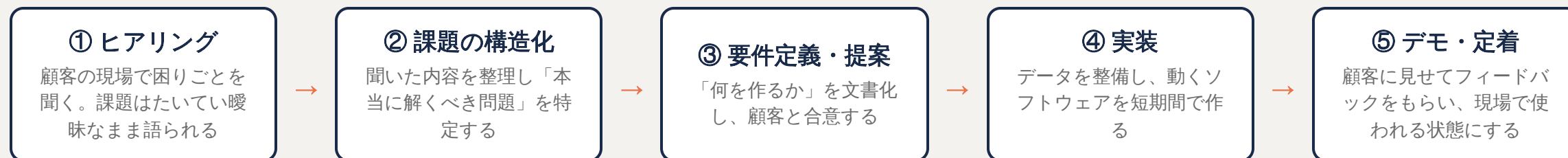
軍事用語の「前方展開」が由来。本社で待つのではなく「顧客の最前線に配置される」ことを表します。

Palantir(パランティア)

米国のデータ分析ソフトウェア企業。FDE という職種を確立したことで有名。同社の FDE は顧客先に常駐し、自社製品(Foundry など)を顧客
の業務に合わせて実装します。

FDE の働き方 — 典型的なプロジェクトの流れ

この流れ全体を一人で回せるようになることが、研修のゴールイメージです



- ポイントは ①～⑤ を高速に何度も回すこと。最初から完璧なものは作らず、小さく作って見せて直す
- ⑤ で「作ったのに使われない」となるのが最大の失敗。現場に定着するまでが FDE の仕事

■ 用語メモ

要件定義

「システムで何を実現するか」を顧客と合意して文書にまとめる工程。あとの工程すべての土台になります。

デモ

デモンストレーション。作ったものを実際に動かして顧客に見せること。口頭説明より何倍も伝わります。

定着(ていちゃく)

作ったシステムが現場で日常的に使われ続ける状態になること。導入(リリース)して終わり、ではありません。

FDE に求められる 3 つのスキル領域

本研修はこの 3 領域をバランスよく鍛える構成になっています

① エンジニアリング

- Python / SQL / Git
- API 設計
- データパイプライン
- LLM・RAG(生成 AI)
- クラウド基礎

② コンサルティング

- 要件ヒアリング
- 課題の構造化
- 提案資料作成
- ステークホルダー管理
- ファシリテーション

③ ドメイン・プロダクト

- 顧客業界の業務知識
- 自社プロダクトの深い理解
(例: Palantir Foundry、生成 AI 基盤)

■ 用語メモ

API	Application Programming Interface。プログラム同士が会話するための「窓口」。例: 天気アプリは気象庁の API を呼び出して天気データを受け取っています。
データパイプライン	バラバラの場所・形式にあるデータを、集めて・変換して・使える形に流し込む一連の処理の流れのこと。
ステークホルダー	プロジェクトの利害関係者。顧客の担当者・その上司・実際に使う現場の人など、立場の異なる関係者全員を指します。
ドメイン	ここでは「事業領域・業界」の意味。製造、金融、医療など、顧客のビジネスの世界のこと。

研修の共通ゴールと修了判定基準

新卒・中途・未経験者、すべてのコースに共通する到達目標

「顧客の曖昧な課題を聞き取り、データとソフトウェアで動くものを短期間で作り、現場に定着させられる」状態になること。

修了判定:以下 3 つの最終アウトプットで判定します

#	判定項目	合格の目安
1	顧客役へのヒアリングから要件定義書を作成できる	課題・スコープ・成功基準が明文化され、顧客役が「これで合っている」と承認できる品質
2	本リポジトリ相当の RAG アプリを自力で改修・機能追加できる	他人の助けなしに、動く機能をプルリクエストとして提出できる
3	成果物を顧客役の前でデモ・プレゼンし、質疑に答えられる	技術に詳しくない相手にも伝わる説明ができ、想定外の質問に誠実に対応できる

■ 用語メモ

RAG(ラグ) Retrieval-Augmented Generation(検索拡張生成)。AI が回答する前に社内文書などを「検索」し、その内容を根拠にして回答を「生成」する仕組み。セクション 2 で図解します。

スコープ プロジェクトで「やること」と「やらないこと」の範囲。ここが曖昧だと、あとから作業が際限なく増えます。

1. FDE (PR) 本リポジトリを知らず、「私が書いたこのコード変更を、チームの正式なコードに取り込んでください」という依頼。取り込む前にレビュー(他人のチェック)を受けるの

SECTION 2

研修の全体設計

受講者のレベルに合わせた 3 つのコースと、全コース共通で使うハンズオン教材(社内情報検索アプリ)を紹介します。

対象者のレベル定義 — 3つのコース

スタート地点が違えば、必要な研修も違う。一律の研修は行いません

コース	想定プロフィール	期間	研修の重点
未経験者	プログラミング経験なし。営業・バックオフィスなどからの職種転換組	16 週間	技術の土台作りに最も時間を割く。「写経 → 模倣 → 自作」の3段階で無理なく積み上げる
新卒	情報系学部出身、または独学でプログラミング基礎あり。実務経験なし	12 週間	「動くコードが書ける」から「実務で通用する」へ。チーム開発の作法とコンサルスキル
中途	エンジニアまたはコンサルタントとして実務経験 3 年以上	6 週間	既存スキルを前提に「自社のやり方」と「不足領域の補完」に絞った短期集中型

 **コースの決め方:**自己申告ではなく、入社時のスキルチェック(簡単なコーディング課題+ケース面接)で客観的に判定します。新卒でもプログラミング未経験に近い場合は未経験者コースからスタートして構いません。

■ 用語メモ

- 写経(しゃきょう)** お手本のコードをそのまま自分の手で書き写して動かす学習法。読むだけより格段に身につきます。
- ケース面接** 「ある企業の売上を伸ばすには?」のようなお題に対し、その場で考えを構造化して答える面接形式。コンサル的思考力を測ります。

ハンズオン教材: 「社内情報検索アプリ」

全コース共通で、実在のリポジトリ `company_inner_search_app2` を教材に使用します

どんなアプリ？

社内の文書(社員名簿・会議の議事録・サービス資料など)に対して、**チャットで質問すると AI が文書を検索して答えてくれる Web アプリ**。生成 AI 活用案件の最も典型的な形です。

技術構成: **Python + Streamlit + LangChain + OpenAI API**(RAG 構成)

主なファイル構成

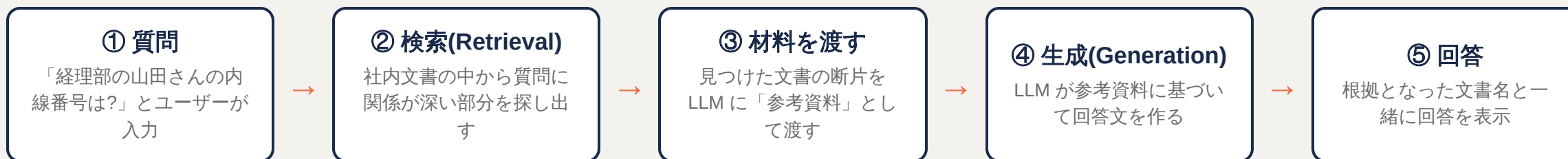
<code>main.py</code>	アプリの入口。画面全体の流れ
<code>initialize.py</code>	起動時の準備(文書の読み込みなど)
<code>components.py</code>	画面の部品(表示まわり)
<code>utils.py</code>	便利な関数のまとめ
<code>constants.py</code>	設定値・定数のまとめ
<code>data/</code>	検索対象の社内文書(教材データ)
<code>test_parantia_ontology.py</code>	Palantir OSDK の検証スクリプト

■ 用語メモ

- リポジトリ** プログラムのソースコード一式を保管する「入れ物」。GitHub 上でチーム全員が同じリポジトリを共有して開発します。
- Streamlit** Python だけで Web アプリの画面が作れるツール。HTML や JavaScript を書かなくてよいので、データ系アプリの試作に最適。
- LangChain** 生成 AI アプリを組み立てるための Python ライブラリ(部品集)。文書の読み込み・検索・AI への質問などの部品が揃っています。
- OpenAI API** ChatGPT を提供する OpenAI 社の AI を、自分のプログラムから呼び出すためのサービス。

基礎知識:RAG の仕組み

教材アプリの心臓部。この図を自分の言葉で説明できることが最初の関門です



- **なぜ検索を挟むのか？** LLM は社内情報を知らないため、そのまま聞くと「もっともらしい嘘(ハルシネーション)」を答えることがある。実在の文書を根拠にさせることで正確さを高める
- 検索の準備として、文書を細かく分割(**チャンク分割**)し、意味を数値化(**Embedding**)して保存しておく。この事前準備が教材アプリの `initialize.py` の役割

■ 用語メモ

LLM

Large Language Model(大規模言語モデル)。ChatGPT の中身にあたる、文章を理解・生成する AI のこと。

ハルシネーション

AI が事実でないことを、さも本当のように答えてしまう現象。「幻覚」が語源。RAG はこの対策として有効です。

チャンク分割

長い文書を数百文字程度の「かたまり(チャンク)」に切り分けること。検索の精度に大きく影響します。

Embedding(埋め込み)

文章の「意味」を数百個の数値の並び(ベクトル)に変換する技術。意味が近い文章は近い数値になるため、「意味で検索」できるようになります。

SECTION 3

未経験者向けコース(16週間)

プログラミング経験ゼロからのスタート。技術の土台作りに最も時間を割き、「写経 → 模倣 → 自作」の3段階で着実に積み上げます。

未経験者コース:16 週間の全体像

フェーズ 1(第1~6週)
エンジニアリング基礎

フェーズ 2(第7~10週)
生成AI・RAG入門

フェーズ 3(第11~13週)
コンサル基礎

フェーズ 4(第14~16週)
総合演習

STEP 1 写経

お手本のコードをそのまま書き写して動かす。まず「動く体験」を積み、コードへの恐怖心をなくす段階。

STEP 2 模倣

お手本を少し変えて別の課題を解く。「この行を変えるとどうなる？」を繰り返し、仕組みの理解に進む段階。

STEP 3 自作

お手本なしでゼロから自分で作る。調べながら自力で完成させる。総合演習はこの段階の集大成。

 **運営メモ:**未経験者は「分からないことが分からない」状態に陥りやすいため、**毎日 15 分の朝会**で詰まりを共有し、30 分悩んだら質問するルール(タイムボックス)を徹底します。

フェーズ 1-a(第 1〜2 週):Python 基礎

すべての土台。ここは焦らず、手を動かす量で勝負します

学ぶこと

- **変数**:データに名前を付けて保存する箱
- **制御構文**:条件分岐(if)と繰り返し(for)
- **関数**:処理に名前を付けて再利用する仕組み
- **クラス**:データと処理をひとまとめにする設計図
- リスト・辞書などのデータ構造、ファイルの読み書き

実践課題

- **演習問題 100 本ノック**:1 問 5〜15 分の小問を大量にこなし、文法を体に覚えさせる
- **CSV 集計スクリプト**:社員名簿の CSV を読み込み、部署ごとの人数を集計して表示するプログラムを作成
- 毎日の成果はすべて Git にコミット(次週の練習も兼ねる)

■ 用語メモ

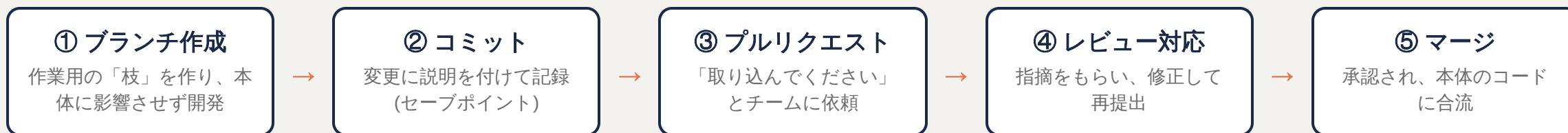
Python(パイソン) 世界で最も使われているプログラミング言語のひとつ。文法がシンプルで読みやすく、AI・データ分析分野の標準語。FDE の主力言語です。

CSV Comma-Separated Values。カンマ区切りのテキストでできた表データのファイル形式。Excel でも開ける、データ受け渡しの定番。

スクリプト 比較的小さな、上から順に実行されるプログラムのこと。

フェーズ 1-b(第 3 週):Git・GitHub・ターミナル操作

チーム開発の共通言語。この週の流れを「毎日」繰り返して体で覚えます



- あわせてターミナル(黒い画面)の基本操作を習得:フォルダ移動、ファイル操作、プログラムの実行
- レビューの受け方も練習:「指摘=人格否定ではない」。コードへの指摘を歓迎するマインドセットを作る

■ 用語メモ

Git(ギット)	ソースコードの変更履歴を記録・管理するツール。「いつ・誰が・何を・なぜ変えたか」が全部残り、いつでも過去に戻れます。
GitHub(ギットハブ)	Gitのリポジトリをインターネット上で共有できるサービス。チーム開発の世界標準プラットフォーム。
ブランチ	「枝」の意味。本体(main ブランチ)から枝分かれして安全に作業し、完成したら合流(マージ)させます。
ターミナル	文字だけでコンピュータに命令する画面。マウス操作より速く正確に作業でき、エンジニアの必須ツール。

フェーズ 1-c(第 4 週):SQL とデータモデリング基礎

FDE の仕事は「データに始まりデータに終わる」。データを扱う共通言語を学びます

学ぶこと

- SELECT / WHERE / JOIN / GROUP BY などの基本文法
- **テーブル設計の基礎**:データをどう表に分けて持つか(例:社員表と部署表を分けて ID でつなぐ)
- 「1つの事実は1か所にだけ書く」という正規化の考え方

実践課題

- 教材リポジトリの `data/` フォルダ(社員・顧客・サービスのデータ)をデータベース化した**サンプル DB**への問い合わせ演習
- 例:「入社3年以内で営業部の社員一覧」「顧客ごとの契約サービス数」を SQL で取り出す

■ 用語メモ

SQL(エスキューエル)	データベースに「こういうデータをください」と依頼するための言語。ほぼすべての企業システムの裏側で使われています。
データベース(DB)	大量のデータを整理して保存し、高速に取り出せるようにした仕組み。Excel の超強力版とイメージすると近いです。
テーブル	データベースの中の「表」。行(レコード)と列(カラム)でできています。
データモデリング	現実の業務(社員、顧客、契約...)を、どのような表とつながりで表現するか設計すること。

フェーズ 1-d(第 5～6 週):Web アプリ基礎

「自分の作ったものが画面で動く」体験で、学習の推進力を作ります

学ぶこと

- **Web の仕組み**:ブラウザとサーバーが HTTP で会話している、という全体像
- **API の概念**:プログラム同士がデータをやりとりする窓口
- **Streamlit 入門**:Python だけで画面(ボタン、入力欄、表)を作る方法

実践課題

- **社内 FAQ 表示アプリをゼロから作る**
 - 質問一覧を表示し、キーワードで絞り込める
 - FAQ データは CSV から読み込む(第 1～2 週の復習)
 - 完成したら同期の前でミニデモ(プレゼンの練習も兼ねる)

■ 用語メモ

- HTTP** ブラウザとサーバーの間の通信ルール。「このページをください(リクエスト)」「はいどうぞ(レスポンス)」のやりとりの決まりごと。
- サーバー** ネットワーク越しにサービスを提供するコンピュータ。Web ページやデータを求めに応じて返してくれます。
- FAQ** Frequently Asked Questions(よくある質問)。

フェーズ 2(第 7～10 週):生成 AI・RAG 入門

いよいよ教材アプリの世界へ。「使う」から「作る側」に回ります

週	テーマ	実践課題
第 7 週	LLM の仕組みとプロンプトエンジニアリング	OpenAI API を呼び出す最小スクリプト(20 行程度)を作成。同じ質問でもプロンプトの書き方で回答がどう変わるか 比較実験レポート を作る
第 8 週	RAG の概念(Embedding / ベクトル検索 / チャンク分割)	教材アプリの <code>initialize.py</code> を読解し、 処理フローを図解して発表 。「起動時に何が起こるかの順番で起きているか」を人に説明できるようにする
第 9～10 週	教材アプリを動かす・改修する	環境構築 → 起動 → 小さな機能改修(例: 回答に参照元のページ番号を表示する)を実装し、プルリクエストで提出。レビューを受けてマージまで完走する

■ 用語メモ

プロンプト	AI への指示文のこと。「あなたは社内ヘルプデスクです。以下の資料に基づいて教えてください」のような文章。
プロンプトエンジニアリング	AI から狙い通りの回答を引き出すために、プロンプトを工夫・設計する技術。
環境構築	プログラムを自分のパソコンで動かすための準備作業。Python のインストール、ライブラリの導入、API キーの設定など。実案件でも最初のつまずきポイントなので研修で体験しておきます。
ベクトル検索	文章を数値(ベクトル)に変換したもの同士を比べ、「意味が近い」文書を探す検索方法。キーワードが一致しなくても意味が近ければヒットします。

フェーズ 3(第 11～13 週):コンサルティング基礎

技術を「顧客の価値」に変換するスキル。FDE のもう半分の武器です

週	テーマ	実践課題
第 11 週	課題の構造化(ロジックツリー / MECE)	「社内の情報検索に時間がかかる」という曖昧な課題を、原因の候補に分解してロジックツリーでドキュメント化する
第 12 週	ヒアリングと要件定義	先輩社員が顧客役を演じる ロールプレイ 。ヒアリング → 議事録作成 → 要件定義書の作成 まで一気通貫で実施
第 13 週	資料作成とプレゼン	要件定義の内容を 経営層向け 5 枚スライド にまとめて発表。「技術の詳細ではなく、相手が知りたいこと」を語る訓練

■ 用語メモ

- ロジックツリー

大きな問題を「なぜ?」「どうやって?」で枝分かれさせて整理する図。原因の見落としや重複を防ぎます。
- MECE(ミーシー)

Mutually Exclusive, Collectively Exhaustive の略。「モレなく、ダブリなく」分類できている状態。構造化の基本原則です。
- ロールプレイ

役割を演じる練習方式。ここでは先輩が「顧客役」になり、本番さながらのヒアリングを体験します。
- 議事録

会議で「何が話され、何が決まり、誰が何をいつまでにやるか」を記録した文書。FDE の新人が最初に任される重要な仕事です。

フェーズ 4(第 14～16 週):総合演習

16 週間の集大成。ヒアリングから実装・プレゼンまでを一人で完走します

課題:「架空の顧客(例:製造業の人事部)の社内文書検索を効率化せよ」

進め方(3 週間)

- 第 14 週:顧客役へのヒアリング → 課題の構造化 → 要件定義書の作成・合意
- 第 15 週:教材アプリをベースに機能追加を実装
例:部署別の検索フィルタ、CSV 社員名簿の横断検索
- 第 16 週:仕上げ → 顧客役へのデモ → 最終プレゼン・質疑応答

サポート体制と評価

- メンター(中途研修修了者など)が週次でコードレビューと壁打ちを実施
- 評価は修了判定基準(要件定義書 / 実装 / デモ)のルーブリックで実施
- 「完璧だが未完成」より「粗くても動いて説明できる」を高く評価する

■ 用語メモ

壁打ち(かべうち) 考えを人に話して整理すること。テニスの壁打ちのように、相手に打ち返してもらいながら思考を磨きます。

コードレビュー 書いたコードを他人がチェックし、バグや改善点を指摘してもらうこと。品質向上と学習の両方に効く、チーム開発の基本文化。

SECTION 4

新卒向けコース(12週間)

プログラミング基礎は習得済みの前提。「動くコードが書ける」と「実務で通用する」の間にある壁を越えるための、実務プラクティスとコンサルスキル中心の構成です。

新卒コース:12 週間の全体像

フェーズ 1(第1~4週)
実務エンジニアリング

フェーズ 2(第5~7週)
プロダクト・プラットフォーム

フェーズ 3(第8~9週)
コンサル実践

フェーズ 4(第10~12週)
模擬プロジェクト

未経験者コースとの違い

- 文法学習はスキップし、初週から**チーム開発の実務フロー**に入る
- 「書ける」の先にある「読める・テストできる・運用できる」を鍛える
- コンサルパートは、要件が曖昧・途中で変わるという**実戦的な設定**で行う

この 12 週間で越えるべき 3 つの壁

- **他人のコードの壁**:自分が書いたことのない大きなコードを読み解く
- **品質の壁**:テスト・ログ・エラー処理まで含めて「完成」とする
- **顧客の壁**:技術の言葉を、顧客のビジネスの言葉に翻訳する

フェーズ 1(第 1～4 週):実務エンジニアリング

教材アプリを「読み・テストし・チューニングする」4 週間

週	テーマ	実践課題
第 1 週	開発環境・チーム開発の作法	教材リポジトリを clone し、 Issue 起票 → ブランチ作成 → PR 提出 → レビュー対応 のサイクルを体験。コミットメッセージの書き方も指導
第 2 週	コードリーディング	<code>main.py</code> / <code>initialize.py</code> / <code>utils.py</code> / <code>components.py</code> の 責務分離 を読み解き、 アーキテクチャ図 を作成して発表
第 3 週	テスト・ログ・エラーハンドリング	<code>test_parantia_ontology.py</code> を参考に、 <code>utils.py</code> の関数へ pytest のテストを追加。ログ設計をレビューし「障害時に原因を追える出力か」を確認
第 4 週	RAG の内部理解	チャンクサイズ・Embedding モデル・検索件数(top-k) を変えて回答品質を比較し、評価レポートを作成。「なんとなく動く」から「根拠を持って調整できる」へ

■ 用語メモ

- Issue(イシュー)

GitHub 上の「課題チケット」。バグ報告や機能要望を 1 件ずつ記録し、作業の単位として管理します。
- 責務分離

「このファイルは画面担当」「このファイルは計算担当」のように、役割ごとにコードを分けて整理する設計の考え方。
- pytest

Python のテストを自動実行するツール。「この関数にこれを入れたらこれが返るはず」を検証するコードを書き、変更のたびに自動チェックできます。
- top-k

検索結果の上位何件を AI に渡すかを決める数値。多すぎるとノイズが増え、少なすぎると根拠が足りなくなります。

フェーズ 2(第 5～7 週):プロダクト・プラットフォーム研修

個々の技術から一段上がり、「データ基盤」というプラットフォームの視点を獲得します

週	テーマ	実践課題
第 5 週	データ基盤とオントロジー概念	Palantir Foundry 等のプラットフォーム概説。 <code>test_parantia_ontology.py</code> を題材に OSDK / オントロジー (Object・Link・Action)の構造 を学習
第 6 週	データパイプライン演習	<code>data/</code> 配下の雑多なファイル(PDF / Word / CSV / 会議議事録)を 構造化データに変換するパイプラインを実装 。実案件の「汚いデータとの格闘」を体験
第 7 週	クラウドデプロイ基礎	アプリを Streamlit Cloud 等に デプロイ し、 環境変数・シークレット管理 を実践。「自分の PC でしか動かない」を卒業する

用語メモ

オントロジー	企業のデータを「モノ(Object:社員・工場・注文など)」「つながり(Link:社員は部署に所属する)」「操作(Action:注文を承認する)」として整理した“業務の地図”。Palantir Foundry の中核概念です。
OSDK	Ontology SDK。オントロジー上のデータをプログラムから操作するための開発キット(SDK = Software Development Kit)。
構造化データ	行と列が決まった、プログラムで扱いやすい形式のデータ。PDF や議事録のような「非構造化データ」から構造化データを作るのが FDE の頻出業務です。
デプロイ	作ったアプリをサーバーに配置し、他の人も使える状態にすること。「配備」の意味。
環境変数・シークレット	API キーやパスワードなどの秘密情報を、コードに直接書かずに外側から渡す仕組み。漏えい事故防止の必須知識。

フェーズ 3(第 8～9 週):コンサルティング実践

きれいな教科書どおりには進まない、実戦形式の 2 週間

実戦型ロールプレイ

- 顧客役の要件が曖昧で、途中で変わる設定で実施
- ヒアリング → 要件定義 → **スコープ交渉**までを体験
「それは今回の範囲外とし、次フェーズのご提案とさせていただきます」と言う練習
- 「技術的にできること」と「顧客が本当に必要なこと」のギャップを埋める**提案書**を作成

シャドーイング(現場観察)

- 先輩 FDE の商談・定例会議に週 2 回以上同席
- 観察ポイントを事前に決めて臨む:
質問の順番/難しい質問のかわし方/宿題の引き取り方/会議の締め方
- 同席後は必ず**気づきメモ**を提出し、メンターと振り返り

■ 用語メモ

- スコープ交渉** プロジェクトの範囲(やること・やらないこと)を顧客とすり合わせる交渉。安請け合いは炎上のもと、断りすぎは信頼低下。バランス感覚が必要です。
- シャドーイング** 先輩に影(shadow)のように同行し、実際の仕事ぶりを間近で観察する学習法。
- 定例(ていれい)** 顧客と定期的に行う進捗確認の会議。週次定例が一般的。FDE の主戦場のひとつです。

フェーズ 4(第 10～12 週):模擬プロジェクト

チームで実在の社内課題に取り組む、研修の総仕上げ

プロジェクト設計

- 3～4 名のチームで実施(役割分担と協働を経験する)
- 題材は**実在の社内課題**(例:営業部の過去提案書検索の効率化)
- 顧客役はマネージャーが担当し、本気でダメ出しをする

進め方と評価

- 週次で顧客役への進捗報告・デモを行い、フィードバックを反映(小さく作って見せるサイクルの実践)
- 最終週:**成果発表会**+個人ごとの振り返り
- **360 度フィードバック**:チームメンバー・メンター・顧客役の全方向から評価を受ける

💡 **運営メモ**:チーム編成は技術力が偏らないように調整します。また「全員が一度は顧客の前で話す」ことをルール化し、実装担当に閉じこもるメンバーを作らないようにします。

■ 用語メモ

360 度フィードバック 上司だけでなく、同僚・後輩・関係者など全方向から評価コメントをもらう仕組み。自分では気づけない強み・弱みが見えます。

SECTION 5

中途向けコース(6週間)

実務経験 3 年以上が対象。既存スキルを前提に、「自社のやり方」の習得と「不足領域の補完」に絞った短期集中型プログラムです。

中途コース:6 週間の全体像と第 1 週(アセスメント)

第 1 週
アセスメント

第 2～3 週
出身別ブリッジ研修

第 4～5 週
実案件シャドーイング(OJT)

第 6 週
独り立ち判定

第 1 週:アセスメント(実力診断)

- コーディングテスト+ケース面接形式のアセスメントで、個人ごとの不足領域を特定
- 結果に基づき、第 2～3 週の研修内容を個別に調整する

第 1 週:オンボーディング課題

- 教材アプリの環境構築 → 起動 → 小改修 PR を初週中に提出
- 目的は開発フローへの習熟確認。「経験者でも当社の進め方に乗れるか」を早期に見る

■ 用語メモ

アセスメント 能力・スキルの客観的な評価・診断のこと。研修を「全員一律」ではなく「個人の穴埋め」にするための出発点。

オンボーディング 新しく加わったメンバーが早く戦力になるための立ち上がり支援プロセス全般。

OJT On the Job Training。座学ではなく、実際の仕事の中で学ぶ育成方法。

第 2～3 週:出身別ブリッジ研修

エンジニア出身とコンサル出身では「足りない半分」が逆。重点を入れ替えて実施します

エンジニア出身の方

技術はある → コンサル力を集中補強

- 課題構造化・提案書作成・顧客折衝のロールプレイ集中訓練
- 価格・スコープ交渉の模擬演習
「技術的にはできます」で安請け合いしない訓練
- 非エンジニアへの説明トレーニング(専門用語の翻訳)

コンサル出身の方

顧客対応はできる → 実装力を集中補強

- Python / Git / RAG 実装のブートキャンプ(未経験者コースの教材を高速で消化)
- 教材アプリへの機能追加を毎週 1 本 PR 提出
例:会話履歴の永続化、検索精度の改善

💡 共通プログラム:自社プロダクト・過去案件のケーススタディを実施。成功案件だけでなく“炎上案件”も教材化し、「何をすると失敗するか」を先に学びます。

■ 用語メモ

ブリッジ研修

今あるスキルと目標(FDE)の間の「橋渡し」をする研修。足りない部分だけを埋める効率重視の設計。

永続化(えいぞくか)

アプリを閉じてもデータが消えないように、ファイルやデータベースに保存すること。

炎上案件

納期遅延・品質問題・顧客との関係悪化などでプロジェクトが混乱状態になった案件のこと。

第 4～5 週:実案件シャドーイング / 第 6 週:独り立ち判定

第 4～5 週:OJT 型実案件シャドーイング

- 実案件に「準メンバー」としてアサインし、議事録作成・小タスクの実装を実際に担当
- 週次でメンター FDE との 1on1を実施
 - 案件での動き方のレビュー
 - 顧客とのコミュニケーションの振り返り

第 6 週:独り立ち判定

- 実案件の 1 テーマを単独で担当し、顧客(または顧客役)向けにデモ・報告を実施
- 判定基準(4 項目)
 - 要件把握の正確さ
 - 実装品質
 - 説明の分かりやすさ
 - 期日管理

■ 用語メモ

- 1on1(ワンオンワン)** 上司やメンターと 1 対 1 で行う定期面談。進捗確認だけでなく、悩みや成長テーマを話す場。
- アサイン** プロジェクトや役割に人を割り当てること。
- 独り立ち** メンターの常時サポートなしで、案件の一部を任せられる状態になること。

SECTION 6

研修の運営設計

研修は「やって終わり」にしない。評価・メンター制度・卒業後フォローまで含めて、育成を仕組み化します。

評価とレベル判定

「なんとなく良かった」を排除し、観点と基準を明文化して評価します

ループリック評価

- 各フェーズ末に**技術 / コンサル / ドメインの 3 軸 × 5 段階**で評価
- 評価基準は事前に受講者へ公開する(何を目指せばいいか明確に)
- 例)技術軸レベル 3: 「レビュー指摘が軽微な修正のみで PR をマージできる」

ポートフォリオの蓄積

- 成果物(**PR・要件定義書・プレゼン資料**)はすべてポートフォリオとして蓄積
- 研修後の**案件アサイン判断**の材料として活用
「この人は検索チューニングの評価レポートが強い → データ品質案件へ」など

■ 用語メモ

ループリック 「観点 × 到達レベル」の表形式で作る評価基準表。評価者による採点のブレを減らし、受講者には成長の地図になります。

ポートフォリオ 自分の成果物を集めた作品集。エンジニアの世界では GitHub 上のコードや資料がそのまま実力の証明になります。

メンター制度 — 「教えることで定着させる」サイクル

体制

- 受講者 2〜3 名に対しメンター FDE 1 名
- 週 1 回の 1on1 + 日常的な PR レビュー
- メンターの役割は「答えを教える」ではなく「考え方を引き出す」
まず自分で調べる → 30 分詰まったら質問、のバランスを作る

育成の循環

- 中途コース修了者が、次期の未経験者・新卒コースのメンターを務める
- 「人に教える」ことが最も深い学習になる(学習定着率が最も高いのは“他人に教える”こと)
- メンター経験自体を FDE のコンサル力(説明力・伴走力)のトレーニングと位置づける

■ 用語メモ

メンター 経験者として学習者に伴走し、助言する役割の人。指示命令する上司とは異なる立場です。

教材リポジトリの使い方(まとめ)と卒業後フォロー

教材	研修での用途
<code>main.py</code> / <code>initialize.py</code> / <code>components.py</code> / <code>utils.py</code> / <code>constants.py</code>	コードリーディング、責務分離・設定管理の学習、改修課題のベース
<code>data/</code> (社員・顧客・サービス・MTG 議事録)	データパイプライン演習、RAG の検索品質評価の題材
<code>test_parantia_ontology.py</code>	テストの書き方、Palantir OSDK / オントロジー概念の学習
<code>requirements_mac.txt</code> / <code>requirements_windows.txt</code>	OS 別の環境構築演習

卒業後のフォロー

- 修了後 3 ヶ月間は月 1 回のフォローアップ会(案件での詰まりを共有・相談)

研修資産の再利用

- 研修課題・模範解答は本リポジトリのブランチとして蓄積し、次期研修で再利用。回を重ねるほど研修が良くなる仕組み

■ 用語メモ

requirements ファイル そのアプリが必要とする Python ライブラリの一覧表。これを使うと誰のパソコンでも同じ環境を再現できます。

APPENDIX

付録:用語集

研修中に出てくる用語を 4 カテゴリーでまとめました。迷子になったらここに帰ってきてください。

用語集 A:IT 基礎

用語	意味
Python	AI・データ分析分野で標準的なプログラミング言語。文法がシンプルで初学者にも学びやすい
SQL	データベースからデータを取り出す・書き込むための言語
データベース(DB)	大量のデータを整理して保存し、高速に取り出せる仕組み
API	プログラム同士がデータをやりとりするための「窓口」
HTTP	ブラウザとサーバーが会話するための通信ルール
サーバー	ネットワーク越しにサービスを提供するコンピュータ
ターミナル	文字コマンドでコンピュータを操作する画面
CSV	カンマ区切りの表データファイル。データ受け渡しの定番形式
クラウド	自前でサーバーを持たず、インターネット経由で借りて使うコンピュータ資源(AWS、Google Cloud など)
デプロイ	作ったアプリをサーバーに配置して、他の人が使える状態にすること
環境変数・シークレット	パスワードや API キーなどの秘密情報を、コードの外側から安全に渡す仕組み

用語集 B:生成 AI・RAG

用語	意味
LLM	大規模言語モデル。ChatGPT の中身にあたる、文章を理解・生成する AI
プロンプト	AI への指示文。書き方の工夫で回答の質が大きく変わる
RAG	検索拡張生成。AI が回答前に文書を検索し、その内容を根拠に回答する仕組み
ハルシネーション	AI が事実でないことを、もっともらしく答えてしまう現象
Embedding(埋め込み)	文章の意味を数値の並び(ベクトル)に変換する技術。「意味で検索」を可能にする
ベクトル検索	Embedding 同士を比較して、意味が近い文書を探す検索方法
チャンク分割	長い文書を検索しやすい小さな「かたまり」に切り分けること
top-k	検索結果の上位何件を AI に渡すかの設定値
LangChain	生成 AI アプリを組み立てるための Python ライブラリ(部品集)
Streamlit	Python だけで Web アプリの画面を作れるツール
オントロジー	企業データを「モノ・つながり・操作」で整理した業務の地図(Palantir Foundry の中核概念)
OSDK	Ontology SDK。オントロジーをプログラムから操作する開発キット

用語集 C:コンサルティング

用語	意味
要件定義	「システムで何を実現するか」を顧客と合意して文書化する工程
スコープ	プロジェクトで「やること・やらないこと」の範囲
ロジックツリー	大きな問題を枝分かれさせて整理する図。原因の見落とし・重複を防ぐ
MECE(ミーシー)	「モレなく、ダブリなく」分類できている状態。構造化の基本原則
ステークホルダー	プロジェクトの利害関係者(担当者・上司・現場の利用者など)
ファシリテーション	会議や議論を円滑に進行し、結論に導く技術
議事録	会議の内容・決定事項・宿題(誰が・何を・いつまでに)の記録
定例	顧客と定期的に行う進捗確認会議
壁打ち	考えを人に話して整理すること
ケース面接	お題に対しその場で考えを構造化して答える面接形式
ドメイン	顧客の事業領域・業界(製造、金融、医療など)

用語集 D:開発プラクティス・育成

用語	意味
Git / GitHub	コードの変更履歴を管理するツール / それをチームで共有するサービス
リポジトリ	ソースコード一式を保管する入れ物
ブランチ	本体に影響を与えず作業するための「枝分かれ」
コミット	変更内容に説明を付けて記録すること(セーブポイント)
プルリクエスト(PR)	自分の変更を本体に取り込んでもらう依頼。レビューを経てマージされる
コードレビュー	他人のコードをチェックし改善点を指摘し合う、品質と学習のための文化
Issue	GitHub 上の課題チケット。作業を 1 件ずつ管理する単位
pytest	Python のテストを自動実行するツール
OJT	実際の仕事の中で学ぶ育成方法(On the Job Training)
1on1	メンター・上司と 1 対 1 で行う定期面談
シャドーイング	先輩に同行して仕事を間近で観察する学習法
ループリック	「観点 × 到達レベル」で作る評価基準表
360 度フィードバック	上司・同僚・関係者など全方向から評価をもらう仕組み



さいごに — FDE への最短ルートは「作って、見せる」

この研修で一貫しているのは「小さく作って、顧客(役)に見せて、直す」というサイクルです。
完璧な準備より、動くものと素直なフィードバックの往復が、FDE を最速で育てます。

運営・カリキュラムへの質問は研修事務局まで | 詳細ドキュメント:docs/fde-training-program.md